

Module 03: Debugging with Codex

Turn runtime errors, stack traces, and broken flows into clear fixes.

Session focus

- Explain failures in plain language
- Trace errors back to real files
- Prompt Codex for fixes you can review

Format

- 1 hour core module
- Hands-on labs: 1-2 hours each

Why Debugging With Codex Matters

Codex helps learners move from confusing symptoms to testable explanations.

Common learner pain

- Stack traces feel noisy
- Root causes hide behind secondary errors
- Fixes are tempting before the failure is understood

Codex advantage

- Summarizes the likely fault
- Points to file and line context
- Suggests remediation steps and patch previews

Learning Objectives

By the end, participants should be able to use Codex as a debugging partner.

Participants practice how to

- Ask Codex to explain runtime errors
- Map stack trace frames back to repository code
- Separate symptom, root cause, and fix
- Harden code after the first fix works

The Debugging Loop

A repeatable workflow keeps Codex grounded in evidence.

Loop

- 1. Reproduce the failure locally
- 2. Capture the exact stack trace or log
- 3. Share relevant files and expected behavior
- 4. Ask for root cause before patch
- 5. Apply, run, and compare the result

Key habit

- Make Codex explain the why, not just the edit.

Lab 3.1: Stack Trace Playground

Learners paste raw traces into a React dashboard and review structured diagnostics.

Build

- Vite React frontend with textarea and sample traces
- Express endpoint: POST /stack-trace
- Diagnostic card with bad line, root cause, fix, and patch preview

Practice

- Submit at least three traces
- Discuss why the suggested fix makes sense
- Compare Codex reasoning with manual debugging

Lab 3.2: Buggy Scripts Rescue

Five intentionally broken Node scripts create a focused repair gym.

Scripts

- calculateBonus.js
- validateUser.js
- emailScheduler.js
- configMerge.js
- retryHandler.js

Workflow

- Run each script manually
- Capture the full error
- Ask Codex for a structured diagnosis
- Apply your own fix after reviewing the patch preview

Lab 3.3: Broken Authorization Project

A realistic React + Express stack turns debugging into a full-service investigation.

Scenario

- correct/ contains the working login flow
- incorrect/ contains intentional backend and frontend issues
- Learners sign in with learner/pass123 or demo/codex

Debug targets

- Missing JSON middleware
- Typo in the login route
- Inconsistent fetch handling
- Network and server logs as evidence

Prompting Pattern for Runtime Failures

Good prompts give Codex evidence, boundaries, and a reviewable output format.

Prompt recipe

- Paste the exact error and stack trace
- State what you expected to happen
- Name the command or UI action that reproduces it
- Include relevant file snippets
- Ask for root cause, minimal fix, and verification steps

Avoid

- Do not ask for a blind rewrite when a narrow fix is possible.

Reviewing Codex Suggestions

The patch preview is a starting point, not the finish line.

Review checklist

- Does the fix address the first failing frame?
- Does it preserve intended behavior?
- Are guards, async handling, and input validation reasonable?
- Can the fix be verified with a command, test, or UI flow?

Learner reflection

- Record where Codex and your manual fix differed.

Wrap-Up Exercise

Document the debugging story, not only the final answer.

Deliverables

- Collect five failing scenarios
- For each one, save the error, root cause, and suggested patch
- Apply a reviewed fix locally
- Compare the before and after behavior

Takeaway

- Codex is strongest when learners provide evidence and verify every repair.